

CSCI-354: Computer Science III

Course Project

Campus Service Management System

Project Weight: 10% of the course grade
Assignment Type: Individual
Programming Language: Python
Submission Deadline: November 23, 2026 at 11:59 PM EST
Presentation Date: November 25, 2026
Instructor: Desta Haileselassie Hagos, PhD
Questions: desta.hagos@howard.edu

Important

This is an **individual course project**. Each student is responsible for designing, implementing, testing, documenting, and presenting their own work. The project is being assigned early so that you have sufficient time to plan and develop it. You are **not expected to implement the entire project immediately**. As we cover additional data structures and algorithms during the semester, you should gradually incorporate them into your project.

1. Project Overview

The goal of this project is to design and implement a **Campus Service Management System** using Python and the data structures and algorithms studied in CSCI-354. A university receives many types of service requests. Examples include:

- information technology support,
- classroom technology assistance,
- building or facilities maintenance,
- housing repair requests, and
- other campus service requests.

Some requests can be processed in the order in which they arrive, while urgent requests may need to be handled before ordinary requests. Your program will manage these requests using appropriate data structures. The objective is not only to create a working program, but also to demonstrate that you understand **why particular data structures are appropriate for particular operations**.

2. Learning Objectives

By completing this project, you should be able to:

- apply multiple data structures to a realistic programming problem;
- select an appropriate data structure for a particular operation;

- implement and use data structures correctly in Python;
- analyze the time complexity of important operations;
- organize a program using classes and functions;
- test and evaluate the correctness of your implementation;
- explain your design and implementation clearly in writing;
- demonstrate and explain your program during a presentation; and
- use Generative AI tools responsibly when appropriate.

3. System Requirements

Your program should provide a menu-driven interface that allows the user to manage campus service requests. A possible main menu could look like the following:

Campus Service Management System

1. Add Normal Request
2. Add Urgent Request
3. Process Next Request
4. Search for Request
5. View Pending Requests
6. View Completed Requests
7. View Statistics
8. Exit

Your interface does not have to look exactly like this example, but the required functionality described below must be available.

3.1. Service Request Information

Each service request should contain appropriate information such as:

- Request ID
- Requester name
- Request category
- Request description
- Priority
- Status

You may include additional fields if they are useful to your design.

Request ID Requirement

Every request must have a **unique Request ID**. The program must detect and reject an attempt to create a second request with the same ID.

The Request ID will be used as the key when requests are organized and searched using the Binary Search Tree.

3.2. Adding a Normal Request

The system must allow the user to create an ordinary service request. Normal requests should be managed using a **queue** so that they can be processed according to their arrival order. For this project, use integer priorities from 1 through 5:

Priority	Meaning
1	Critical
2	High
3	Medium
4	Low
5	Routine

A smaller priority number represents a more urgent request. If two urgent requests have the same priority, the request submitted earlier must be processed first.

3.3. Processing Requests

The system must provide an operation for processing the next request. Use the following policy:

1. If one or more urgent requests are pending, process the urgent request with the highest priority.
2. If no urgent requests are pending, process the normal request that has waited the longest.

If two urgent requests have the same priority, process the request that arrived first. After processing a request:

- update its status from **Pending** to **Completed**;
- remove it from the appropriate pending-request structure; and
- add it to the completed-request history.

Processing Policy

Urgent requests are processed before normal requests. Among urgent requests, priority determines the order. Ties are resolved using arrival order. Normal requests follow FIFO order.

3.4. Completed Request History

Use a **stack** to maintain completed requests in most-recently-completed-first order. The system must support at least one meaningful stack operation, such as:

- viewing the most recently completed request;
- reopening or undoing the most recently completed request; or
- displaying recent completions from newest to oldest.

Displaying the history should not permanently remove its entries unless the selected operation is explicitly intended to do so. You must explain why LIFO behavior is appropriate for the operation you implement.

3.5. Searching for Requests

The system must support searching for a request using its unique Request ID. A **Binary Search Tree (BST)** must be used to organize requests for searching. Each BST entry should associate:

(Request ID, ServiceRequest object)

The Request ID serves as the BST key. The associated **ServiceRequest** object contains the remaining request information. The BST should:

- act as an index of all requests;
- support searching for both pending and completed requests; and
- demonstrate how comparisons between Request IDs guide the search through the tree.

A request should not be removed from the BST when it is completed. Instead, its status should be changed from **Pending** to **Completed**. The queue, binary heap, stack, and BST may contain references to the same **ServiceRequest** objects. This allows a request's status to remain consistent throughout the system.

3.6. Displaying Requests

The system should allow the user to display useful information, including:

- pending normal requests,
- pending urgent requests, and
- completed requests.

3.7. System Statistics

The program should provide a simple summary of the current system. For example, the summary may include:

- total number of requests,
- number of pending requests,
- number of completed requests, and
- number of urgent requests.

4. Required Data Structures

Your project must meaningfully use the following data structures:

1. **Queue**
2. **Stack**
3. **Priority Queue / Binary Heap**
4. **Binary Search Tree**

The purpose of the project is not simply to include these data structures somewhere in the program. Each data structure must perform a meaningful function in the overall system. A possible design is shown below.

Data Structure	Possible Purpose	Important Operations
Queue	Manage ordinary service requests in arrival order	enqueue, dequeue
Stack	Maintain recent completed-request history	push, pop
Binary Heap	Manage urgent requests according to priority	insert, remove highest-priority request
Binary Search Tree	Index all requests using unique Request IDs	insert, search

You may make reasonable design decisions that differ from the examples above, but you must clearly explain and justify those decisions in your final report.

5. Program Design

Your program should use appropriate classes and functions. For example, your implementation may contain classes such as:

```
class ServiceRequest:
    ...

class RequestQueue:
    ...

class PriorityRequestQueue:
    ...

class RequestBST:
    ...
```

Your exact class structure may differ from this example. You should design your program so that each class has a clear responsibility and related operations are organized logically.

6. Algorithm and Complexity Analysis

You must analyze important operations in your program using Big- O notation. Your report should include a table similar to the following:

Data Structure	Operation	Expected Complexity
Queue	Enqueue / Dequeue	$O(1)$
Stack	Push / Pop	$O(1)$
Binary Heap	Insert	$O(\log n)$
Binary Heap	Remove priority item	$O(\log n)$
BST	Search / Insert	$O(h)$

For the BST, you should explain what h represents and discuss how the shape of the tree affects performance. Do not simply provide complexity values. Briefly explain **why** the important operations have those complexities.

7. Python Code Requirements

You must submit the complete working Python source code for your project. Your code should:

- run successfully;
- implement the required functionality;
- use the required data structures meaningfully;
- use classes and functions appropriately;
- use meaningful variable, class, function, and method names;
- be organized and readable;
- contain useful comments where appropriate; and
- handle reasonable user input correctly.

Important: Implementing the Data Structures

The purpose of this project is to demonstrate your understanding of data structures and algorithms. Therefore, the required structures must be implemented as part of your program.

- The **stack** must provide push and pop operations. A Python list may be used as its underlying storage.
- The **queue** must provide enqueue and dequeue operations with FIFO behavior. You may use `collections.deque` as the underlying storage, but you must provide and use your own queue class and queue operations.
- The **binary heap** must be represented using a Python list and must implement the required sift-up and sift-down operations.
- The **BST** must use linked tree nodes and must implement insertion and search using Request IDs as keys.

You may use ordinary Python lists as underlying storage where appropriate. However, you may not replace a required data structure with another built-in structure that avoids demonstrating the concept being assessed. For example:

- using a Python dictionary instead of implementing the BST does not satisfy the BST requirement;
- using `list.sort()` to manage urgent requests does not demonstrate a binary heap; and
- calling `heapq.heappush()` and `heapq.heappop()` without implementing the heap operations does not satisfy the binary-heap requirement.

8. Testing Requirements

You are responsible for testing your program thoroughly. Your tests should demonstrate both the required functionality and the correct behavior of the underlying data structures. At a minimum, test:

- adding multiple normal requests;
- processing normal requests in FIFO order;
- adding multiple urgent requests with different priorities;
- processing urgent requests in priority order;
- processing urgent requests that have equal priorities to verify that arrival order is used to break ties;
- attempting to add a duplicate Request ID and verifying that it is rejected;
- processing when no requests are pending;
- searching for an existing pending Request ID;
- searching for an existing completed Request ID;
- searching for a Request ID that does not exist;
- displaying pending normal and urgent requests;
- displaying completed requests; and
- verifying that the reported system statistics are correct.

Your program should be tested with enough data to demonstrate that the required data structures are working as intended. For each test, your report should identify:

- the operation or behavior being tested;
- the test data or user input;
- the expected result; and
- the actual result produced by the program.

9. Written Report

Submit a detailed report in **PDF format**. The report should be written clearly and should explain your own design and implementation. The report must include the following sections.

9.1. Student Information

Include:

- Full name
- Student ID number
- Howard University email address

- Course: CSCI-354
- Project title

9.2. Introduction and Problem Description

Explain:

- what the program does;
- what problem it is designed to solve; and
- the main functionality provided by the system.

9.3. System Design

Describe how the major parts of the program work together. You may include a diagram if it helps explain your design.

9.4. Data Structures Used

For every major data structure:

- identify the data structure;
- explain where it is used;
- explain why it is appropriate for that operation; and
- discuss important operations and their time complexity.

9.5. Classes

Explain **every class** in your program. For each class, explain:

- its purpose;
- the important data it stores; and
- how it interacts with other parts of the program.

9.6. Functions and Methods

Explain each important function or method in your program. For each one, describe:

- what the function or method does;
- its inputs or parameters;
- what it returns, if anything; and
- how it contributes to the program.

Do not simply paste code and repeat it line by line. Your explanation should demonstrate that you understand the logic of your program.

9.7. Algorithm and Complexity Analysis

Discuss the important algorithms used in the project and analyze the time complexity of major operations. Your analysis should connect the complexity to the data structure being used.

9.8. Testing and Results

Explain how you tested the program and what the tests demonstrate. Include screenshots showing the program running successfully. Screenshots should demonstrate several different operations, such as:

- adding normal requests;
- adding urgent requests;
- processing requests;
- searching for requests;
- viewing completed requests; and
- displaying statistics.

Screenshots should be readable and should clearly show both the user input and the program output.

9.9. Challenges and Lessons Learned

Briefly discuss:

- challenges encountered while developing the project;
- how you addressed those challenges; and
- what you learned from completing the project.

9.10. GenAI Use Statement

Include a short statement describing your use of Generative AI tools. If you used GenAI, identify:

- the tool or tools used;
- what you used them for;
- how you checked the suggestions or generated content; and
- what changes or corrections you made when appropriate.

If you did not use a GenAI tool, simply state that no GenAI tool was used for the project.

9.11. Conclusion

Summarize the completed system and what you learned about applying data structures and algorithms to a programming problem.

10. Code in the Report

You do **not** need to reproduce your entire Python program in the PDF report because the complete source code will be submitted separately. You may include short code excerpts when they help explain an important class, function, algorithm, or design decision. The report should focus primarily on explaining your design, implementation, testing, and understanding.

11. Presentation

Each student will give an individual presentation of approximately **5–10 minutes**. Your presentation should include approximately **5–15 slides**. A suggested organization is:

1. Project overview and problem
2. System design
3. Data structures used
4. Important classes and algorithms
5. Complexity analysis
6. Testing and results
7. Challenges and lessons learned
8. Conclusion

You should also be prepared to give a brief demonstration of your working program. During or after the presentation, you may be asked questions about your code, classes, functions, algorithms, or data structures. You must be able to explain the work you submitted.

12. Responsible Use of Generative AI

Responsible use of Generative AI tools is permitted for this project. Examples of appropriate uses include:

- brainstorming possible approaches;
- asking for explanations of concepts;
- debugging code;
- obtaining suggestions for improving code;
- generating test ideas; and
- receiving feedback on writing.

However, the submitted project must demonstrate your own understanding.

Responsible GenAI Use

You are responsible for understanding and verifying everything you submit, including any code or explanation produced with the assistance of a Generative AI tool. Do not submit AI-generated code that you cannot explain. During the presentation, you may be asked to explain any class, function, method, algorithm, data structure, or section of code in your project.

13. Academic Integrity and Individual Work

This is an **individual project**. The design, implementation, report, and presentation you submit must represent your own work. You may not:

- copy another student's code;
- share your implementation with another student;
- copy an implementation from an online repository;
- submit another person's work as your own; or
- submit AI-generated work that you do not understand.

Similarity between student submissions may be reviewed. You should be prepared to explain any part of your submitted code during your presentation or when asked by the instructor. Failure to demonstrate reasonable understanding of submitted work may affect the project grade and may be addressed under the course academic integrity policy.

14. Submission Requirements

14.1. Submission Format

Submit one compressed `.zip` file containing:

- all required Python source files;
- the project report in PDF format;
- the presentation slides in PDF format; and
- a short `README.txt` file explaining how to run the program.

The program must run from the command line using:

```
python main.py
```

Do not include virtual environments, cache folders, or unrelated files. Make sure that all submitted files clearly identify you and your project.

15. Grading

The course project is worth **10% of the final course grade**. The project will be graded out of **10 points**.

Component	Points
Correctness and required functionality	2.5
Correct implementation and use of required data structures	2.0
Code organization and readability	1.5
Written report and complexity analysis	1.5
Testing and evidence of results (screenshots)	0.5
Presentation and demonstration	1.0
Understanding and the ability to explain submitted work	1.0
Total	10.0

16. What Will Be Evaluated

A successful project is not simply a program that produces output. Your grade will reflect:

- whether the program works correctly;
- whether the required data structures are implemented and used appropriately;
- whether you understand why those data structures were selected;
- whether you can analyze important operations;
- whether your code is organized and understandable;
- whether your report clearly explains your work;
- whether you tested your implementation thoroughly; and
- whether you can explain and demonstrate your own project.

Project Scope

A graphical user interface, database, web application, user accounts, and permanent file storage are not required. A clear command-line program that correctly implements the required data structures and operations is sufficient.

17. Getting Started

You are receiving the project early so that you can work on it gradually throughout the semester. A good way to begin is:

1. Read the complete project specification.
2. Think about what information a service request should contain.
3. Sketch the overall organization of your program.
4. Identify which part of the system will use each data structure.

5. Begin with the concepts that we have already covered.
6. Add and refine components as we study additional data structures and algorithms.
7. Test each component before combining everything into the final system.

Final Reminder

Do not wait until the final week to begin the project. The project is designed to be developed gradually as we progress through the course. Ask questions early if any requirement is unclear.